Solution: Subarray Sum Equals K

Problem Statement

Given an array `nums` containing `n` integers and integer `k`, return the total number of `subarrays` having `sum` equal to `k`.

A `subarray` is defined as a contiguous `non-empty` sequence of the array elements.

Examples

Example 1:

- **Input**: `nums = [1, 2, 3], k = 3`
- **Expected Output**: `2`
- **Justification**: There are two subarrays that sum to 3: `[1, 2]` and `[3]`.

Example 2:

- **Input**: `nums = [10, 2, -2, -20, 10], k = -10`
- **Expected Output**: `3`
- **Justification**: Three subarrays sum up to -10: `[10, 2, -2, -20]`, `[2, -2, -20, 10]`, and `[-20, 10]`.

Example 3:

- **Input**: `nums = [5, 1, 2, -3, 4, -2], k = 3`
- **Expected Output**: `2`
- **Justification**: There are two subarrays that sum to 3: `[2, -3, 4]`, and `[1, 2]`.

Solution

To solve this problem, we'll employ a technique that involves using a hashmap to efficiently track the cumulative sum of elements as we iterate through the array. The core idea is that if the cumulative sum up to two indices, say `i` and `j`, differs by the target value `k`, then the sum of the elements lying between `i` and `j` is `k`.

The algorithm will iterate through the array, calculating the cumulative sum at each step. We then check if `(cumulative sum - k)` is present in the hashmap. If it is, it means there exists a previous cumulative sum such that the difference between the current sum and that sum equals `k`, indicating a valid subarray. We add the count of these occurrences to our total. Additionally, we keep updating the hashmap with the count of each cumulative sum encountered. This approach is effective as it allows us to find the required subarrays in a single pass through the array, making it efficient in terms of time complexity.

Step-by-step Algorithm

- Initialize a variable `count` to 0 and a hashmap `cumulativeSumFrequency` with a key-value pair (0:1) to handle edge cases.
- Iterate through the array `nums` while keeping track of the `cumulativeSum`.
- For each element `num` in `nums`:
  - Add `num` to `cumulativeSum`.
  - Check if `(cumulativeSum - k)` is in `cumulativeSumFrequency`. If it is, add its frequency to `count`.
  - Update `cumulativeSumFrequency` by incrementing the count of `cumulativeSum`.
- Return the value of `count`.

Algorithm Walkthrough

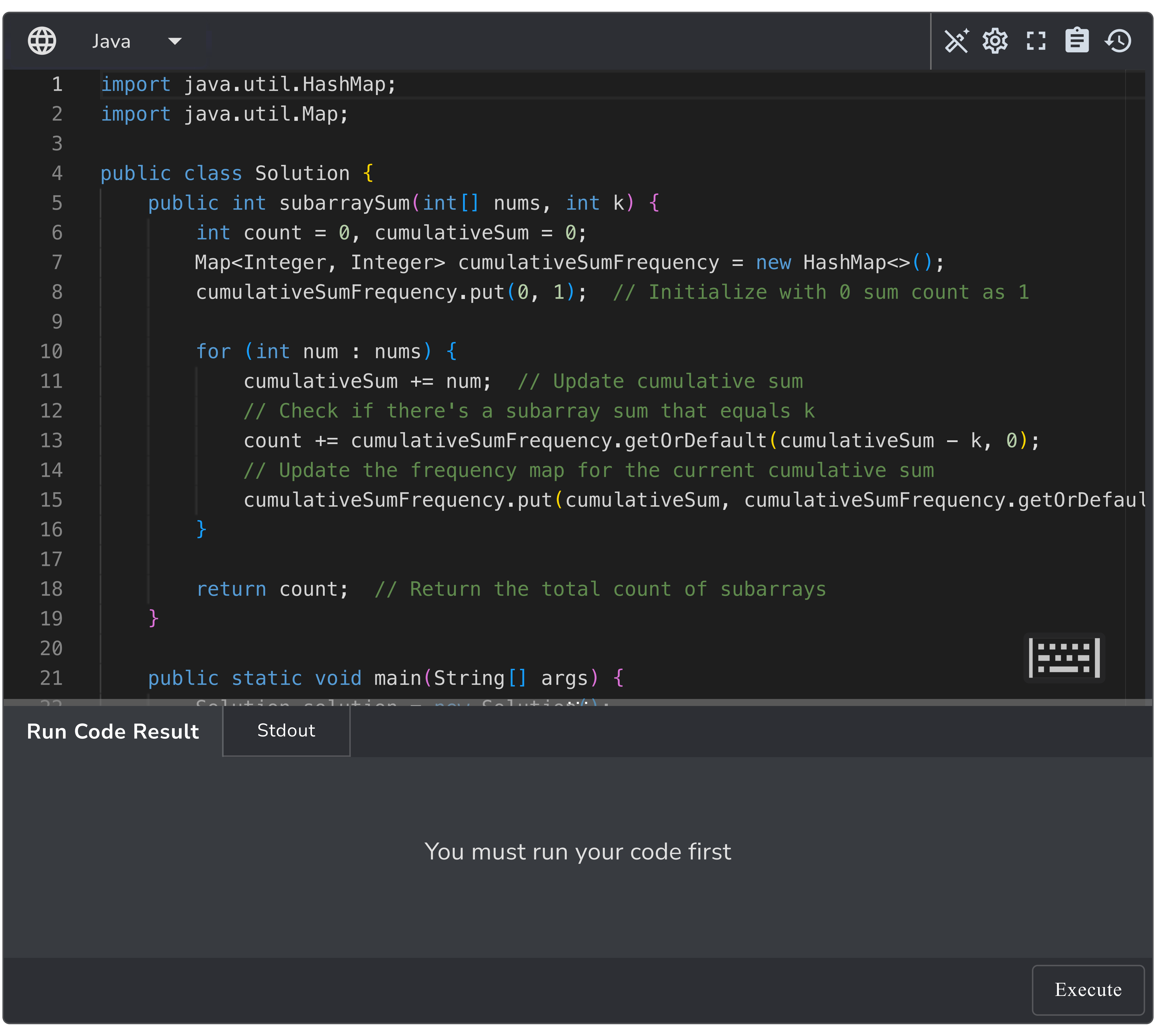
Let's walk through the algorithm with Example 2: `nums = [10, 2, -2, -20, 10], k = -10`.

- Start with `cumulativeSum = 0`, `count = 0`, `cumulativeSumFrequency = {0: 1}`.
- For `num = 10`: `cumulativeSum = 10`. `(cumulativeSum - k) = 20` is not in `cumulativeSumFrequency`. Update `cumulativeSumFrequency` to `{0: 1, 10: 1}`.
- For `num = 2`: `cumulativeSum = 12`. `(cumulativeSum - k) = 22` is not in `cumulativeSumFrequency`. Update `cumulativeSumFrequency` to `{0: 1, 10: 1, 12: 1}`.
- For `num = -2`: `cumulativeSum = 10`. `(cumulativeSum - k) = 20` is not in `cumulativeSumFrequency`. Update `cumulativeSumFrequency` to `{0: 1, 10: 2, 12: 1}`.
- For `num = -20`: `cumulativeSum = -10`. `(cumulativeSum - k) = 0` is in `cumulativeSumFrequency`. Add 1 to `count`. Update `cumulativeSumFrequency` to `{0: 1, 10: 2, 12: 1, -10: 1}`.
- For `num = 10`: `cumulativeSum = 0`. `(cumulativeSum - k) = 10` is in `cumulativeSumFrequency` with frequency 2.

Add 2 to `count`. Update `cumulativeSumFrequency` to `{0: 2, 10: 2, 12: 1, -10: 1}`.

- Final `count` is 3.

Code



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    public int subarraySum(int[] nums, int k) {
        int count = 0, cumulativeSum = 0;
        Map<Integer, Integer> cumulativeSumFrequency = new HashMap<>();
        cumulativeSumFrequency.put(0, 1);  // Initialize with 0 sum count as 1

        for (int num : nums) {
            cumulativeSum += num;  // Update cumulative sum
            // Check if there's a subarray sum that equals k
            count += cumulativeSumFrequency.getOrDefault(cumulativeSum - k, 0);
            // Update the frequency map for the current cumulative sum
            cumulativeSumFrequency.put(cumulativeSum, cumulativeSumFrequency.getOrDefaul
        }

        return count;  // Return the total count of subarrays
    }

    public static void main(String[] args) {
```

You must run your code first

Complexity Analysis

- **Time Complexity**: `O(n)`, where n is the length of the input array. This is due to a single loop through the array and constant-time hashmap operations.
- **Space Complexity**: `O(n)`, where n is the length of the input array. The primary space usage is the hashmap storing the cumulative sum frequencies, which in the worst case can grow to the size of the array.

← Previous
Subarray Sum Equals K (Medium)

Next →
Next Permutation (Medium)

☐ Mark as Completed